

Lab 1: Calibration

Analysis Procedure

NB: Labs 1 and 2 are largely derived from prior activities developed by Dr. Nate McCrady for use with the UM 0.4m observatory.

1 Background and Goals

In this lab, we are going to practice calibrating a CCD. Calibrating a detector in astronomy means removing as many of the systematic errors as we possibly can and characterizing random errors in the data collection process. Fortunately, we're working with a well-designed, commercially available detector of a format that is very common. That means we know roughly what the systematic errors in our detector should be. Of course, all measurements have some random error, but again, this type of CCD is well-understood so we know where errors should be arising.

This lab will also serve as an introduction to Lab 2 – we'll be using the same equipment, so we will need to apply the calibration we develop in Lab 1 to Lab 2.

At the end of this lab, you will have characterized the QHY 163M's bias level and will have applied your calibration information to a real science image. Optionally, you may also calculate the QHY 163M's gain.

2 Our Detector

We will be using a QHY 163M detector, pictured in Figure 1. It is a back-illuminated 3522×4656 pixel CMOS detector, with square pixels each having side length $3.8\mu\text{m}$. That means the detector is made up of a total of 16,398,432 pixels, each of which is a square of area $14.4\mu\text{m}^2$. This is like having 16,398,432 tiny photomultiplier tubes, all counting photons. The camera has an electric rolling shutter and can have exposure times from 60 microseconds to 30 minutes, although our exposure times will likely be in the tenths-of-second to one minute time range.

The detector is monochrome – meaning it doesn't see color. It measures only total intensity, so to obtain color information, we have to place a colored filter in front of the CMOS to block out certain wavelengths of light.

Cameras like the QHY 163M are most sensitive when the sensor is cool, so the QHY 163M has a built in thermoelectric cooler, which we control along with the camera in MaxIm DL. This cooler can reduce the temperature to up to -40°C below ambient temperature, although we'll be using it at -20°C . (See CMOS instructions).

When the shutter is open, photons strike each pixel, causing the emission of photoelectrons. The QHY 163M counts and digitizes the photoelectron information. Technically, the sensor sees an analog measurement of charge, but these charges are converted into an integer number of "counts." The data are reported in analog-to-digital units (ADU) which are integers (functionally: counts) and are stored in a FITS file. One count does not necessarily equal one electron; it just means some interval of charge – more on that later.

An important note: the QHY 163M has a 12 bit analog to digital converter (ADC). Therefore, the maximum count value it can record is $2^{12} - 1 = 4095$. At counts higher than 4095, the detector is "saturated." We don't want to record data past the point where we saturate the detector as we're not actually getting any new info (and it can put stress on the detector).



Figure 1: The QHY 163M CMOS detector.

MaxIm DL inconveniently converts the 12-bit raw data to a 16-bit FITS file for storage. It's not actually improving the data; it's just doing a silly conversion. It writes out maximum values of 65,504. This means we need to convert our data back to the real values, following the expression

$$\text{Real Data} = \text{Recorded Data}/16 + 1. \quad (1)$$

This can be done after loading the fits data into our python notebook/script.

The QHY 163M is controlled using a computer equipped with MaxIm DL. MaxIm DL is unfortunately not an open source software, so we'll all use the same computer to control the camera. After you save your data, you can offload them from the control computer to your own laptop. Expect to collect a few GB of data – too much to email. (So bring an external hard drive or large-ish flash drive when you observe).

When you are observing, don't forget to choose a naming scheme for your files that you'll be able to understand later. You do not want to open your data folder and see "my_fits_001.fits" through "my_fits_145.fits." Keep a log of the observations you've taken including file name, exposure time, date, and (where applicable) object.

3 Sources of noise in the QHY 163M

3.1 Systematic Noise Sources

As we will discuss in class (and your textbook discusses in chapter 9), there are three sources of systematic error in our data. Two are additive: bias and dark. One is multiplicative: flat.

We will measure each of them to eventually apply them to our science images.

3.1.1 Bias

This source of noise is intentionally added into the CCD by the manufacturers. Analog-to-digital converters themselves are just electronic components: they have no idea what problem they're being

applied to. Therefore, they default to a Gaussian distribution of possible values centered on 0.

But we know that counts is not quantity that will ever be negative. Therefore, all those possible negative values are just reducing our “dynamic range” or number of possible values by a factor of two. CCD manufacturers do know what problem they’re addressing, so they introduce an offset during manufacturing, so that the full dynamic range is usable. That means that the “average 0” point for our CMOS is not at 0.

To correct for bias, **collect** about 20 “bias” images using the “bias” setting in the MaxIm DL Expose tab. These images will be taken with the lens cover on and will have functionally 0 exposure time. That is because we don’t need to wait for counts to measure these counts – this is our “zero point.”

Calculate an average bias image by averaging per pixel across all 20 individual bias images. This will reduce random error (see 3.2 versus just showing a single bias image. (Hint: You want to average along the image axis. For an array `bias` with shape `[20, 3522, 4656]`, this looks like `np.mean(bias,axis=0).`)

Calculate the *median* value of the average bias image and discuss any large-scale variations across your average bias image.

3.1.2 Dark Images

A dark image is similar to a bias image, in that it is not an image of the sky. It is also taken with the lens cover on the camera, but it is different because the dark image is of a similar duration to the science images. It is measuring how many counts the detector electronics will count in the absence of real signal.

For the exposure times you’re likely to use in this lab, this may be very similar to the bias image.

After you complete your science images, **collect** a series of 10 dark images by covering the camera lens and taking images with the same exposure times as your science images. Take one set for each exposure time you use for your science images. **Calculate** an average dark in the same way you determined an average bias image, but repeat this process for each exposure time.

3.1.3 Twilight Flats

The QHY 163M is an array detector, which means each of the more than 16 million pixels in the QHY 163M is a tiny photomultiplier tube. Each one of those photomultiplier tubes varies in its exact sensitivity to light. This phenomenon is called the *quantum efficiency* of the pixel.

Quantum efficiency varies from pixel to pixel at the few percent level, and for many applications this can be a big problem. Suppose you’re measuring the brightness of a distant star and that star happens to land entirely on a low efficiency pixel. Your brightness measurement will ultimately be incorrect.

The best way to correct for quantum efficiency is to conduct an experiment where each pixel is exposed to even illumination (known as a flat field) where each pixel should be measuring the same thing and to measure the differences between those pixels.

It’s difficult to do this in the lab, but easy to do this by staring at a twilight sky. We will therefore use our twilight flats to prepare a “normalized flat.”

We collect many sky images (let’s say N – each one will be $i = 0, 1, 2, \dots$) which have the same exposure time but varying brightness. We measure the value of a pixel z_i at position (x, y) , and we know the average bias at that position $b(x, y)$. Our textbook uses the expression

$$z_i^n(x, y) = \frac{z_i(x, y) - b(x, y)}{\text{MODE}(z_i - b)}, \quad (2)$$

where $\text{MODE}(z_i - b)$ is the most common value for a pixel, with the measured bias subtracted off. Sadly for us, this formulation takes up an obscene amount of memory.

Therefore, we're going to approximate it slightly differently, by replacing the denominator with the median pixel value across the flat after bias correction: $z_{i,\text{med}} = \text{MEDIAN}(z_i - b)$.

$$z_i^n(x, y) = \frac{z_i(x, y) - b(x, y)}{z_{i,\text{med}}}, \quad (3)$$

This will give us a single normalized flat, which will be very susceptible to random error. But if we make a series of many normalized flats known as a stack and take the median value, we will have a more reliable normalized flat. In your lab report **explain** why we might use the median instead of the mean.

Collect a series of twilight flats, as described in the accompanying “Observing Procedure.”

Calculate a stack of normalized flats and a median normalized flat. If you find your computer does not like analyzing the entire stack (highly likely), you may choose a sub-set of the stack from which to create your median normalized flat. In your report, include an image of your normalized flat that demonstrates the pixel-to-pixel variation in the flat. This image may be only a small region of your normalized flat.

3.2 Random Error

We will operate under the assumption that we are in the linear regime and therefore that our dominant source of random noise is the read noise instead of the Poisson shot noise.

The read noise is a consequence of the fact that the ADC is converting an analog voltage into a digital count. The readout is itself a counting experiment, and the detector will report a statistical distribution of answers centered on the mean value. This creates uncertainty in the final value for each pixel.

Naively, we might estimate the read noise by taking the standard deviation of average bias image. Shouldn't our science images just start counting from a “zero point” at our bias value? But this approach is stymied by the fact that the bias in our CCD is not totally constant – there is some structure evident towards the edges of the image.

Instead, we will use a common astronomer trick known as “difference imaging.” If we have an image with a fixed pattern, but we are interested only in *variations* in the image, we can take 2 identical images and subtract one from the other. This will leave only the variations.

Apply the method of difference imaging, using two of your bias images. Confirm that the average value of the differenced image is close to 0 (as the fluctuations should not be systemically varying in either direction). The pixel-to-pixel variation of this differenced image is the read noise.

Show the read noise graphically by **plotting** a histogram of your difference image. **Calculate** the read noise

$$\sigma_{\text{read}} = \frac{\sigma_{\text{diff}}}{\sqrt{2}}. \quad (4)$$

Justify the factor of $\sqrt{2}$ in your report using error propagation arguments.

4 Correcting Science Images

Now, we have the tools we need to create calibrated science images. This is actually quite a simple procedure! We can't remove the random noise in our data – that's intrinsic to taking data – but we can remove the systematic noise sources. That's what we measured with our bias, dark, and twilight images.

This process in astronomer's parlance is called “data reduction,” and it's a huge task. Fortunately, many of the data sources we use later in the semester (and that you may use for your final project) are already reduced. When a file says it's been reduced, this is what they mean: they took the appropriate measurements to determine systematics and applied them to the raw data.

So let's do it! The formula for a reduced image of the sky $S_i^R(x, y)$ is

$$S_o^R(x, y) = \frac{S_i(x, y) - \frac{t_s}{t_d}d(x, y) - b(x, y)}{f_i^n(x, y)}, \quad (5)$$

where S_i is the uncorrected image of the source, $d(x, y)$ is a representative or average dark (with bias correction applied – otherwise we'd subtract the average bias counts off twice), $b(x, y)$ is an average bias, and $f_i^n(x, y)$ is a normalized flat.

Apply this formula to your two science images, creating calibrated images. **Explain** why each type of calibration image appears where it does in this expression.

This image would be expected to have a random noise equal to σ_{read} , but it rarely makes sense to present a value for noise in an image – we'd keep that number in our back pocket for further analysis with this image.

5 Determining CCD Gain

You do not need to complete this analysis for Lab 1, but if you do it will be worth extra credit (5 points). You will eventually need to complete this analysis for Lab 2, but we won't be able to discuss Least Squares Fitting in depth until too late to require you to use it for Lab 1.

The gain of the detector is a multiplicative factor that determines how many electrons constitute one count (ADU) for the ADC. Usually, it's not ideal to have $g = 1e^-/\text{ADU}$; each count or ADU will represent several electrons. The largest number of counts that can be recorded by an ADC depends on the “bit depth” or number of bits in the ADC.

Our ADC has 12 bit digitization, so it can record 2^{12} values (from 0 to 4095). Each pixel in the detector is capable of storing more than one electron (sometimes even *many* more than one electron) before writing out. For example, if each pixel can record 20,000 electrons, we might want to have each ADU represent several electrons ($20,000e^-/4095\text{ADU} \sim 4.9$). In that example, we'd say that the gain is $g = 4.9e^-/\text{ADU}$, allowing us to count more than 4095 electrons before saturating.

This does result in a loss of precision, as we no longer have individual electron measurements. But on the other hand, we also detect more total electrons. And as we have Poisson uncertainty running around, we “win” in improvement in signal-to-noise ratio.

Measuring the gain is a little tricky. We'll begin by taking a wide range of flat images at different illumination levels (as we already did measuring the twilight sky). Define your illumination “levels” by looking at your total range of counts (compare your first flat to your last flat) and seeing how the number of counts per pixel changes. Select 5-10 values that represent your “levels” from the peak counts to the minimum counts. For example, if my initial flat has on average 1100 counts and my final flat has an average of 50 counts, I might select “levels” at 1000, 750, 500, 250, 100 and 50.

(You will likely find that the brightness of the sky fell off exponentially: flat 1 is more different from flat 2 than flat 52 is from flat 53.)

For each illumination “level,” pick two flat images, which we’ll call A and B . These images should both have an average count level at approximately your “level” – they will likely be consecutive images. Bias correct each of them. Now choose a small area of the detector that is very uniform in illumination, probably a square that’s 50-100 pixels per side. Measure the mean signal in each subarray (call them F_A and F_B). Calculate the ratio of these mean signal levels ($r = F_A/F_B$), which should be about 1. Multiply image B by ratio r to set the mean signal level to be the same between the images without changing the noise structure.

Now create a difference image (call it D but don’t confuse it with a dark) by subtracting $A - r \cdot B$. This should kill off our pixel-to-pixel variations while leaving behind the Poisson variations between the two similar illumination images. Now select the same sub-array you used before and measure the variance of that sub-array (σ_D^2). Our assumption is that the variance is the same in image A and image B ; error propagation would tell us that the variance σ_A^2 or σ_B^2 are both $\sigma_D^2/\sqrt{2}$.

We want to make 5-10 “data points” – average illumination and variance for a flat image. The data points will be (F_A, σ_A^2) . Plot these data.

By definition,

$$N_e = g \cdot N_{\text{ADU}}, \quad (6)$$

where N_e is the number of electrons recorded and N_{ADU} is the recorded number of counts. The number of recorded electrons is a counting experiment and therefore a Poisson process. That means that the uncertainty is $\sigma_e = \sqrt{N_e}$ or $\sigma_e^2 = N_e$. With error propagation, confirm to yourself that $\sigma_e^2 = g^2 \sigma_{\text{ADU}}^2$ and that the slope of your plot is

$$g = \frac{N_{\text{ADU}}}{\sigma_{\text{ADU}}^2}. \quad (7)$$

Use least squares fitting to calculate a value for g with uncertainty.

6 Your report should include..

Plots:

- Average bias image
- Median normalized flat
- Difference image histogram
- Three corrected science images

Final calculated quantities:

- Median value from average bias image
- Read noise of the detector (in ADU)

Bonus Options:

- Detector Gain (5 points): A variance vs. average count plot, a gain value from your least squares fit, and a brief explanation of your process.